1. Bevezetés

Röviden a projektről

A **ForgeX** vizsgaremek egy olyan komplex webes platform, amelyet a sportolni vágyók és az egészségtudatos életmódot követők számára fejlesztettük ki. Az oldal célja, hogy egyetlen felületen kínáljon minden olyan eszközt és információt, amely a fizikai fejlődéshez és az egészséges életvitel kialakításához szükséges.

A projekt sokrétűségét a különböző funkciók adják: a felhasználók a **BMI-kalkulátor** segítségével pontos képet kaphatnak aktuális testi állapotukról, majd az interaktív **Gyakorlatok** és **Anatómia** menüpontok révén szakmai alapokra helyezhetik edzéseiket. Az oldal lehetőséget biztosít **személyi edzőkhöz** történő időpontfoglalásra, a **Webshopon** keresztül pedig prémium sportruházat és táplálékkiegészítők beszerzésére is mód nyílik. A ForgeX nem csupán egy információs portál, hanem egy interaktív segítőtárs, amely a méréstől a gyakorlati megvalósításon át a felszerelés biztosításáig támogatja a felhasználót céljai elérésében.

Regisztráció

1. Funkcionális leírás

A regisztrációs modul célja az új felhasználók biztonságos rögzítése a rendszerben. A felület modern, reszponzív és több csatornás regisztrációt tesz lehetővé.

Főbb funkciók:

- **Adatbevitel:** Teljes név, e-mail cím, telefonszám és jelszó bekérése.
- **Interaktív UI:** Jelszó láthatóságának kapcsolása.
- **Validáció:** Üres mezők és formátumok ellenőrzése.
- **Firebase integráció:** Felhasználó létrehozása hitelesítéssel és kiegészítő adatok tárolása adatbázisban.
- **Visszajelzés:** Sikeres regisztráció esetén átirányítás, hiba esetén hibaüzenet megjelenítése.

2. Frontend felépítés (HTML & CSS)

2.1. Szerkezeti felépítés (signUp.html)

Az oldal szemantikus HTML5 elemeket használ. A regisztrációs űrlap logikailag csoportosított beviteli mezőkből áll. Minden mező mellett FontAwesome ikonok segítik a vizuális tájékozódást.

2.2. Megjelenés és Design (signIn.css & style.css)

A design a **Glassmorphism** stílusjegyeit hordozza.

- **Üveg-effekt:** A .container osztály backdrop-filter: blur(12px) és rgba alapú háttérszínek használatával ér el áttetsző, modern hatást.
- **Reszponzivitás:** Media query-k segítségével (@media (max-width: 767px)) az oldal mobilalkalmazás-szerű élményt nyújt: a kártya kitölti a képernyőt, a gombok mérete pedig az érintőképernyőhöz igazodik.
- **Interakció:** A :focus-within szelektorral emeltem ki az aktív beviteli mezőket, javítva a felhasználói élményt (UX).

3. Programozási logika (signUp.js)

A kliensoldali logika aszinkron JavaScript folyamatokra épül, amely közvetlenül kommunikál a Google Firebase felhőalapú szolgáltatásaival.

3.1. Felhasználói interakciók kezelése

A kódban implementáltam egy jelszó-megjelenítési funkciót, amely a jelszó mező type attribútumát módosítja (text vs password).

3.2. Regisztrációs folyamat

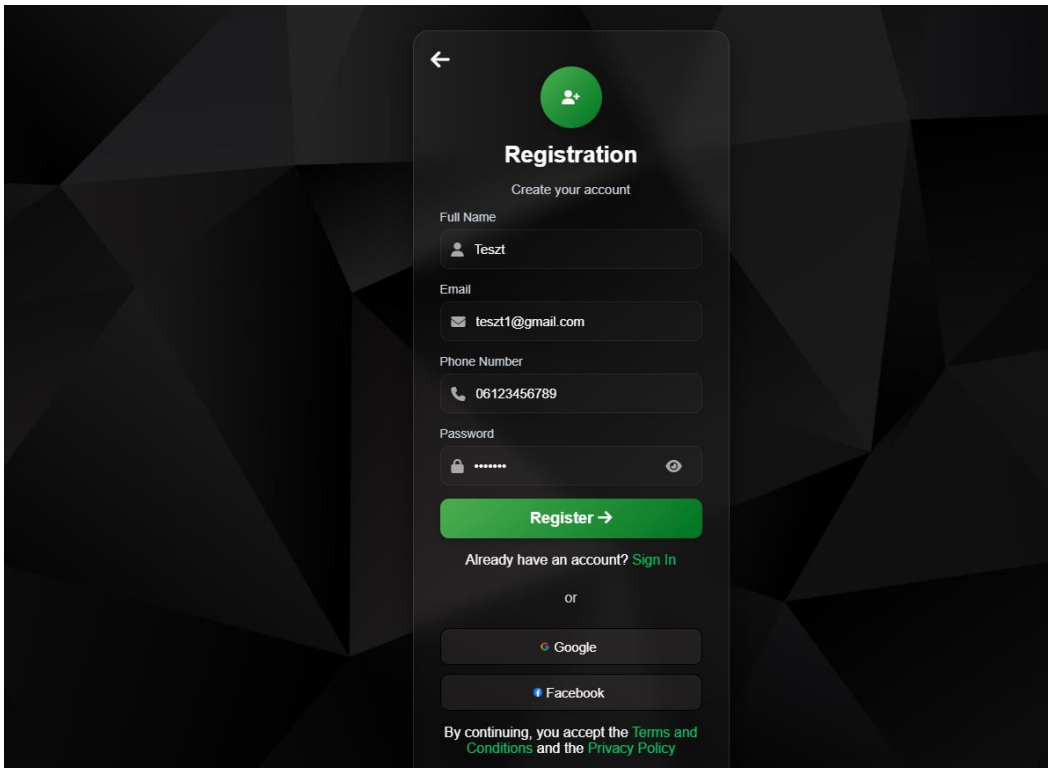
A regisztráció egy aszinkron eseménykezelőn (submit) keresztül fut le:

1. **Alapértelmezett működés gátlása:** Az event.preventDefault() megakadályozza az oldal újratöltését.
2. **UI állapotkezelés:** A küldés gomb letiltásra kerül, hogy elkerüljük a többszöri regisztrációt.
3. **Firebase Authentication:** A createUserWithEmailAndPassword függvénnyel létrehozzuk a fiókot.
4. **Firestore Adatbázis:** A sikeres hitelesítés után kapott egyedi azonosítóval (user.uid) létrehozunk egy dokumentumot a users gyűjteményben, ahol a kiegészítő adatokat (név, telefonszám) tároljuk.

3.3. Hibakezelés

A folyamatot try-catch blokkba zártam.

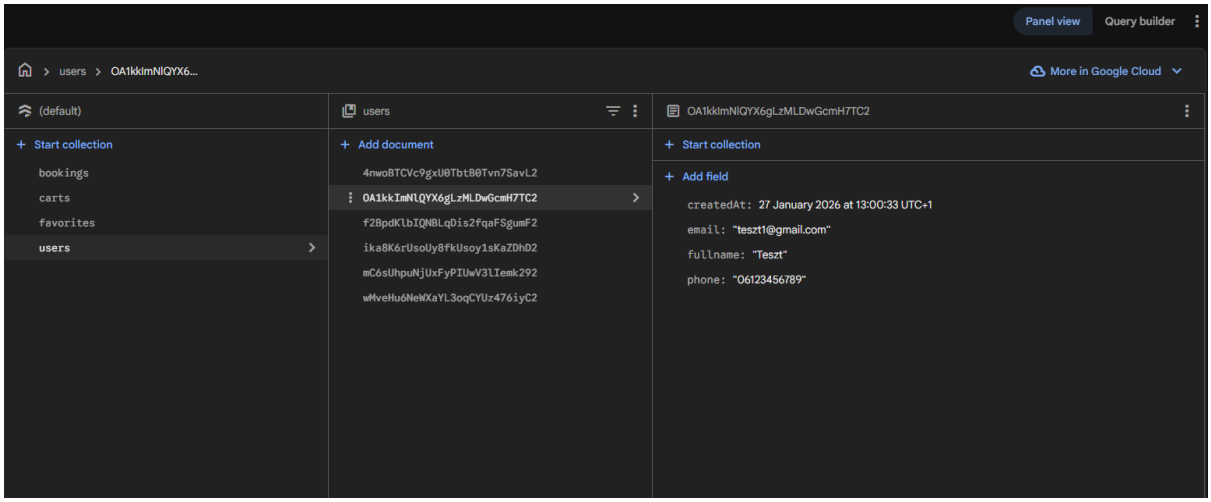
- **Specifikus hiba:** Ha az e-mail már foglalt (auth/email-already-in-use), a rendszer figyelmezteti a felhasználót.
- **Általános hiba:** Minden egyéb esetben a hiba kódja a konzolra kerül, a felhasználó pedig egy általános értesítést kap.



Kitöltött regisztráció oldal.

4. Tesztelés

Tesztelt funkció	Művelet	Elvárt eredmény	Állapot
Kötelező mezők	Üresen hagyott név/email	A böngésző jelzi, hogy a mező kitöltése kötelező.	**Megfelelt**
Email formátum	"pelda@gmail" (hiányzó .hu/.com)	A HTML5 validáció hibát jelez az email formátum miatt.	**Megfelelt**
Jelszó láthatóság	Kattintás a `togglePassword` ikonra	A jelszó karaktereinek láthatóvá kell válniuk (`type="text"`).	**Megfelelt**
Sikeres regisztráció	Érvényes adatok megadása	Az adatok bekerülnek a Firestore-ba, és átirányítás történik a `signIn.html`-re.	**Megfelelt**
Reszponzivitás	Ablakméret csökkentése (375px)	Az oldal igazodik a ablak méretéhez	**Megfelelt**



A felhasználó adatainak tárolása (fiók létrehozásának dátuma, email, teljes név, telefonszám) a Firebase adatbázis “users” táblájában.

Bejelentkezés

1. Funkcionális leírás

A bejelentkezési modul feladata a már regisztrált felhasználók azonosítása és beléptetése a rendszerbe. A felület kiemelt figyelmet fordít a biztonságra és a felhasználói visszajelzésekre.

Főbb funkciók:

- **Hitelesítés:** E-mail cím és jelszó páros ellenőrzése a Firebase Auth rendszerében.
- **Jelszókezelés:** Interaktív jelszó-megjelenítési lehetőség és link az elfelejtett jelszó helyreállításához.
- **Hibakezelés:** Egyedi modális ablakban megjelenő hibaüzenetek a felhasználó tájékoztatására.
- **Navigáció:** Gyorselérés a főoldalhoz (house ikon) és a regisztrációs felülethez.

2. Frontend felépítés (HTML & CSS)

2.1. Felületi elemek (signIn.html)

A felület a regisztrációs oldallal konzisztens vizuális nyelvet használ.

- **Szemantikus felépítés:** A bejelentkezés egy #login-form azonosítójú űrlapon keresztül történik.
- **Ikonográfia:** FontAwesome ikonok segítik az adatok azonosítását (boríték az e-mailhez, lakat a jelszóhoz).
- **Vissza gomb:** Egy abszolút pozicionált ház ikon biztosítja, hogy a felhasználó bármikor visszatérhessen a főoldalra.

2.2. Design és Reszponzivitás (signIn.css)

A megjelenés alapja itt is a **Glassmorphism** technika.

- **Kártya design:** A .container osztály backdrop-filter: blur(12px) és finom szegélyek használatával emelkedik ki a háttérből.
- **Input validáció vizuálisan:** A :focus-within szelektor segítségével a beviteli mezők zöld keretet kapnak fókusz esetén, visszajelzést adva az aktív mezőről.
- **Mobil optimalizálás:** 767px alatti szélességnél a kártya szélessége 95%-ra nő, a belső térközök (padding) pedig csökkennek a jobb olvashatóság érdekében.

3. Programozási logika (signIn.js)

A kliensoldali script moduláris felépítésű, aszinkron hívásokkal kezeli a hálózati kommunikációt.

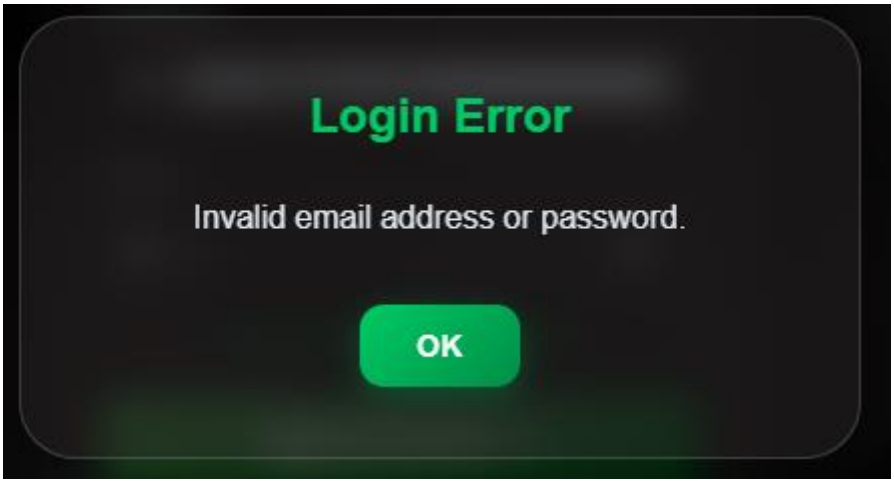
3.1. Jelszó láthatóságának vezérlése

A felhasználói élmény javítása érdekében implementáltam egy váltókapcsolót (togglePasswordButton). Kattintáskor a script megvizsgálja a jelszómező aktuális típusát, majd váltja azt password és text között, egyúttal cseréli az ikont is (fa-eye / fa-eye-slash).

3.2. Hitelesítési folyamat

A beléptetés a signInWithEmailAndPassword aszinkron függvénnyel történik:

1. **Esemény elkapása:** Az űrlap elküldésekor megakadályozzuk az alapértelmezett frissítést.
2. **Firebase kommunikáció:** A megadott adatokat elküldjük a Firebase szervereinek.
3. **Sikeres belépés:** Pozitív válasz esetén a window.location.href segítségével átirányítjuk a felhasználót az index.html-re.
4. **Hibakezelés:** Hiba esetén a script kategorizálja a hiba kódját (pl. hibás hitelesítési adatok vagy túl sok kísérlet), és a saját fejlesztésű forgeXModal segítségével értesíti a felhasználót, tehát egy saját stílusú üzenetet kap a felhasználó.



Ezen a képen látható a az üzenet amit a felhasználó kap valamilyen bejelentkezési hiba esetén (pl.: hibás jelszó megadása)

4. Tesztelési napló

Tesztelt eset	Művelet	Elvárt eredmény	Állapot
Üres mezők	Bejelentkezés gomb megnyomása adatok nélkül.	A böngésző "mező kitöltése kötelező" figyelmeztetést ad.	**Megfelelt**
Hibás jelszó	Helyes e-mail címhez tartozó hibás jelszó megadása.	Megjelenik a "Invalid email address or password" hibaüzenet.	**Megfelelt**
Nem létező fiók	Olyan e-mail cím megadása, ami nincs a rendszerben.	A Firebase biztonsági okokból ugyanazt az általános hibaüzenetet adja, mint rossz jelszónál.	**Megfelelt**
Brute-force védelem	Sorozatos (10+) próbálkozás hibás adatokkal.	Aktiválódik az auth/too-many-requests hiba, a rendszer ideiglenesen letiltja a belépést.	**Megfelelt**
Sikeres belépés	Érvényes e-mail és jelszó páros megadása.	Azonnali átirányítás a főoldalra.	**Megfelelt**

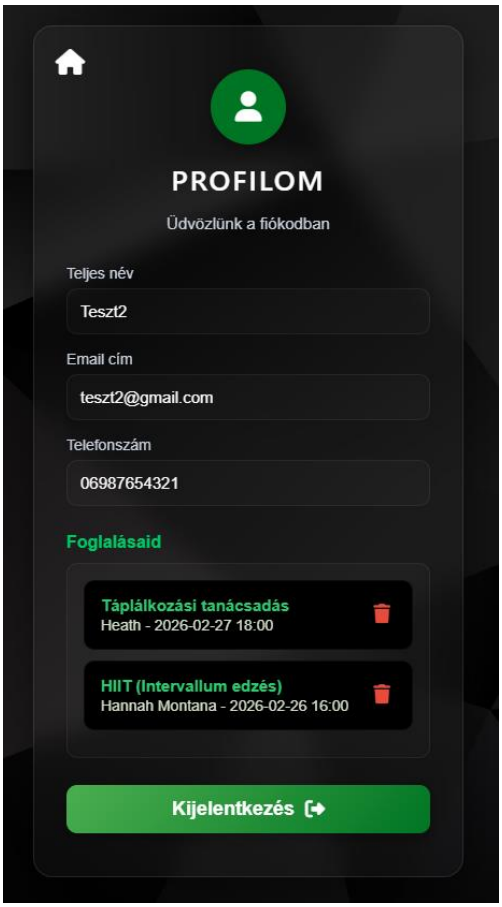
Profil

1. Funkcionális leírás

A profil oldal célja, hogy a bejelentkezett felhasználók megtekinthessék személyes adataikat, valamint kezelhessék (megtekinthessék és törölhessék) az aktív szolgáltatás-foglalásaikat.

Főbb funkciók:

- Munkamenet ellenőrzés: Csak hitelesített felhasználók érhetik el; illetéktelen hozzáférés esetén automatikus átirányítás a bejelentkezéshez.
- Profilmegjelenítés: A Firestore adatbázisból kinyert adatok (név, email, telefon) dinamikus betöltése.
- Foglaláskezelés: A felhasználó saját foglalásainak listázása és azok törlési lehetősége.
- Kijelentkezés: A biztonságos kilépés és a munkamenet lezárása.



Aktív profil oldal ami kiírja a felhasználó adatait valamint a foglalt időpontokat.

2. Frontend felépítés (HTML & CSS)

2.1. Felületi elemek (profil.html)

A felület a letisztultságra és az adatok olvashatóságára fókuszál.

- **Információs kártya:** Egy központi elemben helyezkedik el minden adat, elkerülve a felesleges navigációt.
- **Dinamikus konténerek:** A #user-bookings div szolgál befogadóként a JavaScript által generált foglalási elemek számára.

2.2. Design és Megjelenés (profil.css)

- **Glassmorphism kártya:** A korábbi modulokkal azonos vizuális stílus (blur, félig áttetsző border).
- **Foglalási tételek:** A .booking-item elemek vizuálisan elkülönülnek a háttértől, flexbox segítségével pozicionálva az adatokat és a törlés gombot.
- **Interaktív elemek:** A törlés gomb (.delete-booking-btn) piros kiemelést és hover-effektust kapott, jelezve a művelet destruktív jellegét.

3.1. Auth State Listener (Állapotfigyelő)

Az onAuthStateChanged figyeli, hogy van-e bejelentkezett felhasználó. Ha nincs user objektum, a rendszer nem engedi betölteni az adatokat, hanem visszaküldi a felhasználót a bejelentkező oldalra.

3.2. Adatlekérdezés (Firestore Read)

Két külön aszinkron folyamat zajlik:

1. **Profil adatok:** A doc és getDoc függvényekkel lekérjük a users gyűjteményből az adott UID-hoz tartozó dokumentumot.
2. **Foglalások:** A query és where feltételek használatával szűrjük a bookings gyűjteményt, hogy a felhasználó **csak a saját** foglalásait láthassa.

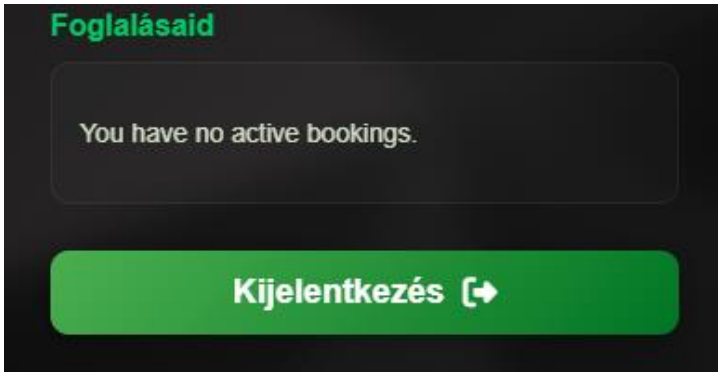
3.3. Dinamikus DOM generálás és Törlés

A foglalások listáját a JavaScript generálja (createElement). Minden tételhez egy egyedi data-id-val ellátott törlés gomb tartozik.

- **Megerősítés:** A törlés nem történik meg azonnal; a forgeXModal aszinkron hívásával egy megerősítő kérdést teszünk fel a felhasználónak.
- **Törlés (Firestore Delete):** Jóváhagyás után a deleteDoc eltávolítja az adatot, majd a lista automatikusan újratöltődik.

4. Tesztelési napló (Profil)

Tesztelt eset	Művelet	Elvárt eredmény	Állapot
Jogosulatlan hozzáférés	Belépés az URL közvetlen beírásával (bejelentkezés nélkül).	A rendszer észleli a hiányzó Auth-ot és átirányít.	**Megfelelt**
Üres foglalási lista	Felhasználó megnyitja a profilt foglalás nélkül.	Megjelenik a "You have no active bookings." üzenet.	**Megfelelt**
Foglalás törlése	Törlés gomb megnyomása + "Mégse".	A modális ablak bezárul, az adat nem törlődik.	**Megfelelt**
Sikeres törlés	Törlés gomb megnyomása + "Ok".	Az elem eltűnik a listából és a Firestore adatbázisból is.	**Megfelelt**
Kijelentkezés	Kijelentkezés gomb megnyomása.	A munkamenet lezárul, átirányítás a főoldalra.	**Megfelelt**



Üres foglalási lista

Shop

1. Funkcionális leírás

A Shop modul a webáruház egyik központi felülete, ahol a felhasználók böngészhetik a kínálatot, szűrhetnek kategóriák szerint, és kosárba helyezhetik a kiválasztott termékeket.

Főbb funkciók:

- **Dinamikus Katalógus:** Termékek megjelenítése grid (sorokból és oszlopokból álló rácsban elhelyezett elemek) elrendezésben, árral és leírással.
- **Keresés és Szűrés:** Valós idejű szöveges keresés és kategória alapú (Ruházat, Kiegészítők) szűrés.
- **Rendezés:** Ár szerinti növekvő és csökkenő sorrendbe rendezés.
- **Termék Részletek (Modal):** Kattintásra megnyíló ablak, ahol méret, szín választható, és több kép található.
- **Kosár Szinkronizáció:** Bejelentkezett felhasználók esetén a kosár tartalma a Firestore-ba is mentődik (felhő alapú mentés).
- **Kedvencek:** Termékek megjelölése kedvencként, szinkronizálva a profiloldallal. A szív ikonra kattintva a termék a kedvencek közé kerül ahol később a sorárba lehet tenni az adott terméket.

2. Frontend felépítés (HTML & CSS)

2.1. Felületi struktúra (shop.html)

- **Side-Filter:** Egy rögzített oldalsáv (mobil nézetben összezárható panel), amely a keresőt és a kategória-választót tartalmazza.
- **Products-Grid:** Flexibilis rácsrendszer, amely automatikusan igazodik a képernyőmérethez (3, 2 vagy 1 oszlop).

2.2. Design és UX (shop.css)

- **Responsive Side-Filter:** Különleges figyelmet kapott a szűrő mobil nézete. A moveSideFilterToContent JS funkció áthelyezi a szűrőt a termékek fölé mobil eszközökön, a CSS pedig asztali gépen rögzített oldalsávként jeleníti meg.
- **Kártya animációk:** A termékkártyák lebegő hatást (transform: translateY(-8px)) kaptak a hover eseményre, ami interaktívabbá teszi a böngészést.
- **Glassmorphism:** A háttér elmosása (backdrop-filter) biztosítja az egységes modern megjelenést minden felületen.

3. Programozási logika (shop.js)

3.1. Keresés és Szűrési logika

A szűrés nem igényel szerveroldali kérést, így rendkívül gyors:

- A filterProducts() függvény minden billentyűleütésre lefut.
- Összeveti a PRODUCT_DATA objektum kategóriáját és a beírt szöveget a kártyák tartalmával.
- A nem egyező elemeket a display: none tulajdonsággal rejti el

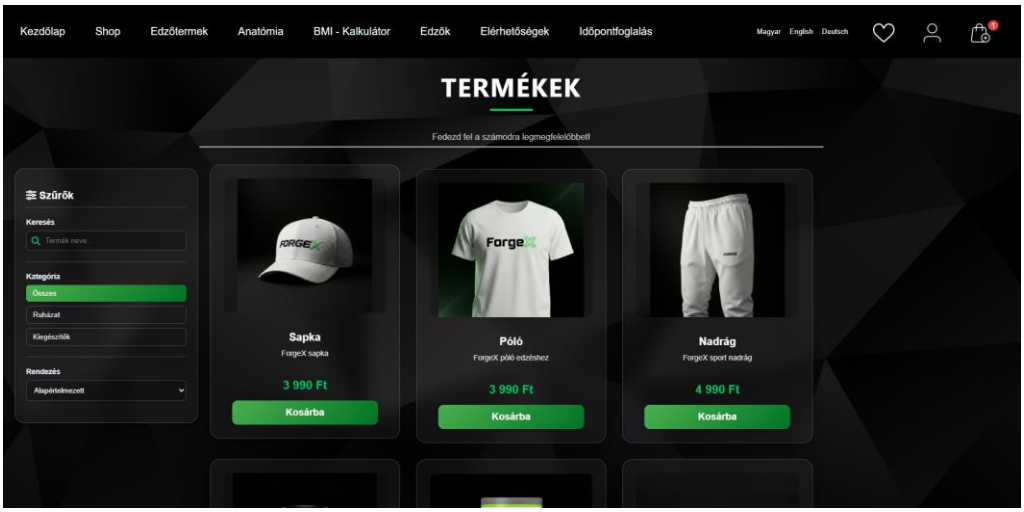
3.2. Kosár és Kedvencek kezelése

- **Firestore szinkronizáció:** Az onAuthStateChanged figyeli a bejelentkezést. Ha a felhasználó belép, a syncCartFromFirestore() letölti a korábbi kosarát, a saveCartToCloud() pedig menti az új módosításokat.

3.3. Dinamikus Modal Galéria

A modal nem csak statikus adatokat mutat, hanem:

- Kezeli a szín- és méretválasztást.
- **Intelligens képváltás:** Ha a termék több képpel rendelkezik (pl. fekete/fehér póló), a szín kiválasztásakor az updateProductImage() automatikusan frissíti a modalban látható képet.



4. Tesztelési napló

Tesztelt eset	Művelet	Elvárt eredmény	Állapot
Valós idejű keresés	"Póló" beírása a keresőbe.	Csak a póló termékkártya marad látható a gridben.	**Megfelelt**
Kategória szűrés	"Ruházat" gombra kattintás.	A kiegészítők (shaker, protein) eltűnnek.	**Megfelelt**
Ár szerinti rendezés	"Olcsó elöl" kiválasztása.	A Fehérje szelet (990 Ft) kerül az első helyre.	**Megfelelt**
Kosár számláló	Termék hozzáadása a gridből.	A fejlécben lévő kosár ikon feletti szám azonnal 1-re nő.	**Megfelelt**
Modal interakció	Kattintás a Sapka kártyára.	Megnyílik a modal a leírással, választható méretekkel.	**Megfelelt**
Felhő szinkron	Belépés után termék kosárba tétele.	Firestore "carts" gyűjteményben megjelenik a rekord.	**Megfelelt**

Kedvencek

1. Funkcionális leírás

A Kedvencek modul célja, hogy a felhasználók elmenthessék a számukra érdekes termékeket későbbi megtekintésre vagy gyors vásárlásra. A modul szoros integrációban áll a Shop és a Kosár modulokkal.

Főbb funkciók:

- **Dinamikus listaépítés:** A Kedvencek oldal automatikusan legenerálja a termékkártyákat a mentett adatok alapján.
- **Gyors vásárlás:** A kedvencek közül a termékek egyetlen kattintással a kosárba helyezhetők anélkül, hogy vissza kellene navigálni a Shop oldalra.
- **Eltávolítás:** A szív ikonra kattintva a termék azonnal kikerül a listából, és a nézet frissül.
- **Üres állapot kezelése:** Ha nincsenek mentett termékek, a rendszer segítő üzenetet jelenít meg a felhasználónak.

2. Frontend felépítés (HTML & CSS)

2.1. Felületi szerkezet

A felület a Shop modul rácsrendszerét (products-grid) használja újra a vizuális konzisztencia érdekében.

- **Dinamikus konténer:** A #favorites-list div szolgál a JavaScript által generált kártyák helyőrzőjeként.
- **Állapotjelző:** A #no-favs elem alapértelmezetten rejtett, és csak akkor jelenik meg, ha a kedvencek tömb üres.

2.2. Design és Megjelenés (favourite.css)

- **Speciális gombok:** A .fav-actions konténer egyedi elrendezést biztosít: a "Kosárba" gomb nagyobb hangsúlyt kap, míg a törlés (szív) ikon mellette helyezkedik el.
- **Interakció:** A törlés gomb (.fav-remove) hover állapotban pirosra vált, vizuálisan jelezve az eltávolítás funkciót.
- **Reszponzivitás:** Mobilon a gombok egymás alá rendeződnek a kényelmesebb érintés érdekében.

3. Programozási logika

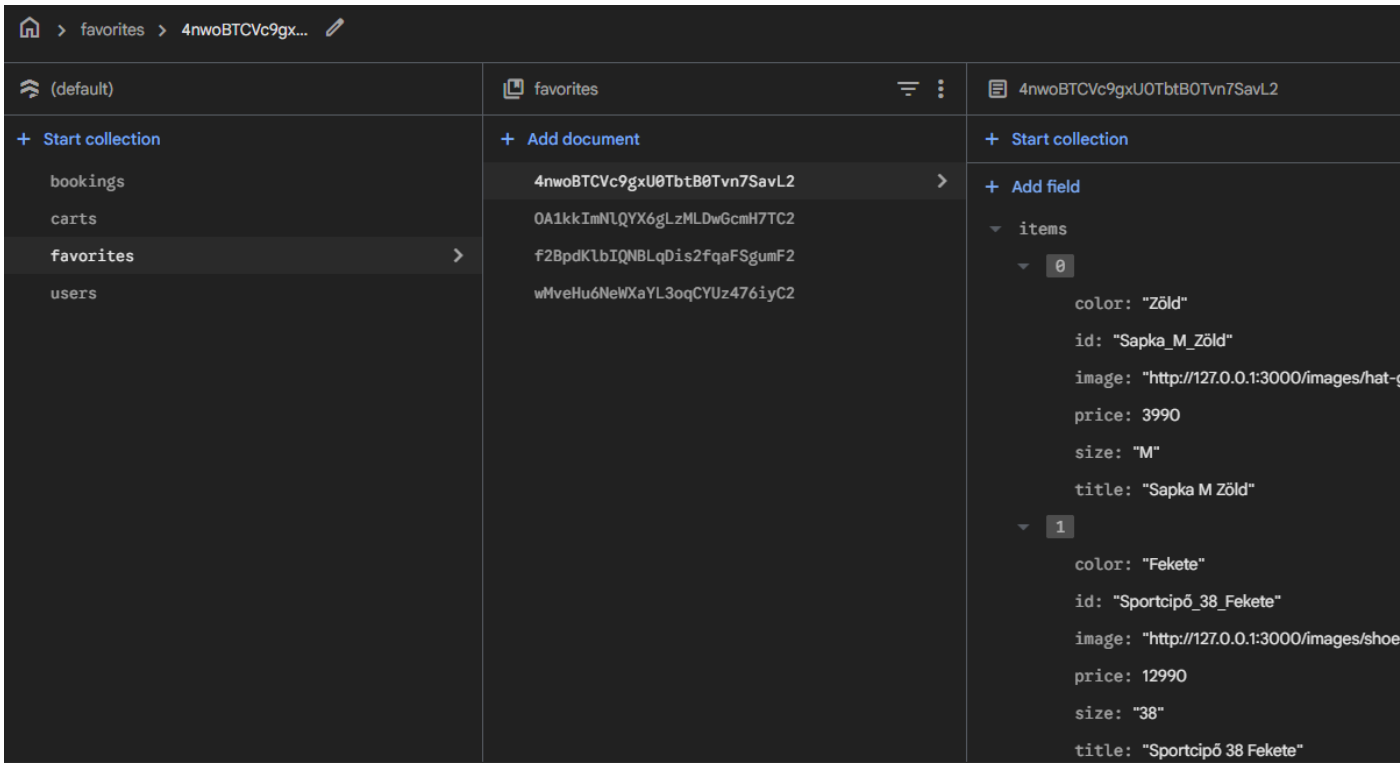
3.1. Adatkezelési folyamat

A modul központi szerepet játszik az adatok szinkronizálásában:

1. **Betöltés:** Az onAuthStateChanged eseménykor a rendszer lekéri a Firestore-ból a mentett tételeket.
2. **Megjelenítés:** A renderFavorites() függvény végigmegy a tömbön, és minden elemhez létrehozza a HTML struktúrát (DOM manipuláció).
3. **Kereszt-funkcionalitás:** Az addToCartFromFavorite() függvény nemcsak a helyi kosár-tömbbe teszi bele a terméket, hanem azonnal elindítja a saveCartToCloud() folyamatot is.

3.2. Felhő-szinkronizáció (CRUD)

- **Mentés (Create/Update):** A setDoc parancs a teljes kedvencek tömböt feltölti a felhasználó egyedi azonosítója (currentUserId) alá.
- **Törlés (Delete):** A removeFavoriteById() frissíti a helyi listát, majd aszinkron módon szinkronizálja a változást az adatbázissal.



Kedvencek tábla az adatbázisban

4. Tesztelési napló

Tesztelt eset	Művelet	Elvárt eredmény	Állapot
Lista betöltése	Kedvencek oldal megnyitása elmentett termékekkel.	A termékek helyesen megjelennek képpel, árral és címmel.	**Megfelelt**
Kosárba tétel	"Kosárba" gomb megnyomása a kedvenceknél.	Megjelenik a forgeXModal sikeres üzenete, a kosár számláló nő.	**Megfelelt**
Elem eltávolítása	Szív ikonra kattintás a listában.	A kártya azonnal eltűnik, a "Nincsenek kedvencek még" üzenet megjelenik, ha ez volt az utolsó elem.	**Megfelelt**
Kijelentkezés hatása	Kijelentkezés a profilnál, majd visszalépés.	A LocalStorage törlődik, a kedvencek lista kiürül.	**Megfelelt**
Felhő integritás	Elem törlése, majd oldalfrissítés.	Az elem az adatbázisból is törlődött, így frissítés után sem jön vissza.	**Megfelelt**

Kosár

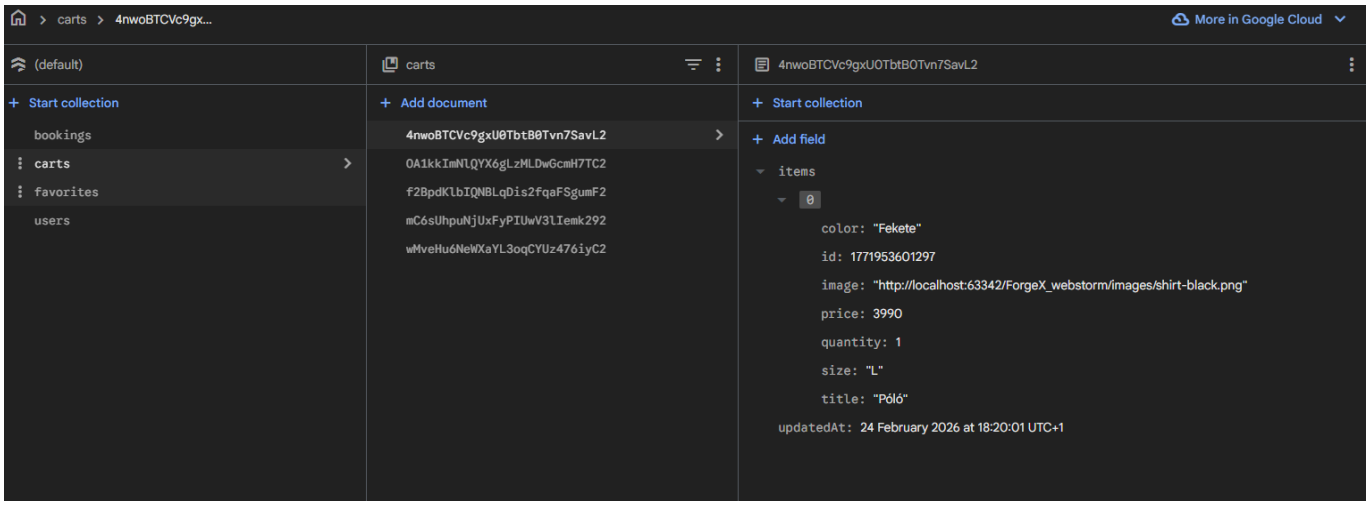
1. Funkcionális leírás

A Kosár modul lehetővé teszi a felhasználók számára a kiválasztott termékek áttekintését, a mennyiségek és paraméterek (méret) ellenőrzését, valamint a nem kívánt tételek eltávolítását a fizetés előtt.

Főbb funkciók:

- **Dinamikus kosár-megjelenítés:** A tárolt tételek automatikus listázása képpel és adatokkal.
- **Valós idejű árkalkuláció:** A végösszeg automatikus újraszámítása minden módosítás (törlés) után.
- **Törlési funkció:** Tételek eltávolítása a kosárból, amely azonnal frissíti a helyi tárolót és az adatbázist is.
- **Vásárlási ösztönzés:** Üres kosár esetén tájékoztató üzenet, teli kosár esetén pedig a "Pénztár" gomb megjelenítése.

A kosár adatait itt is a Firebase adatbázisában azon belül is a “carts” táblában tároltuk.



2. Frontend felépítés (HTML & CSS)

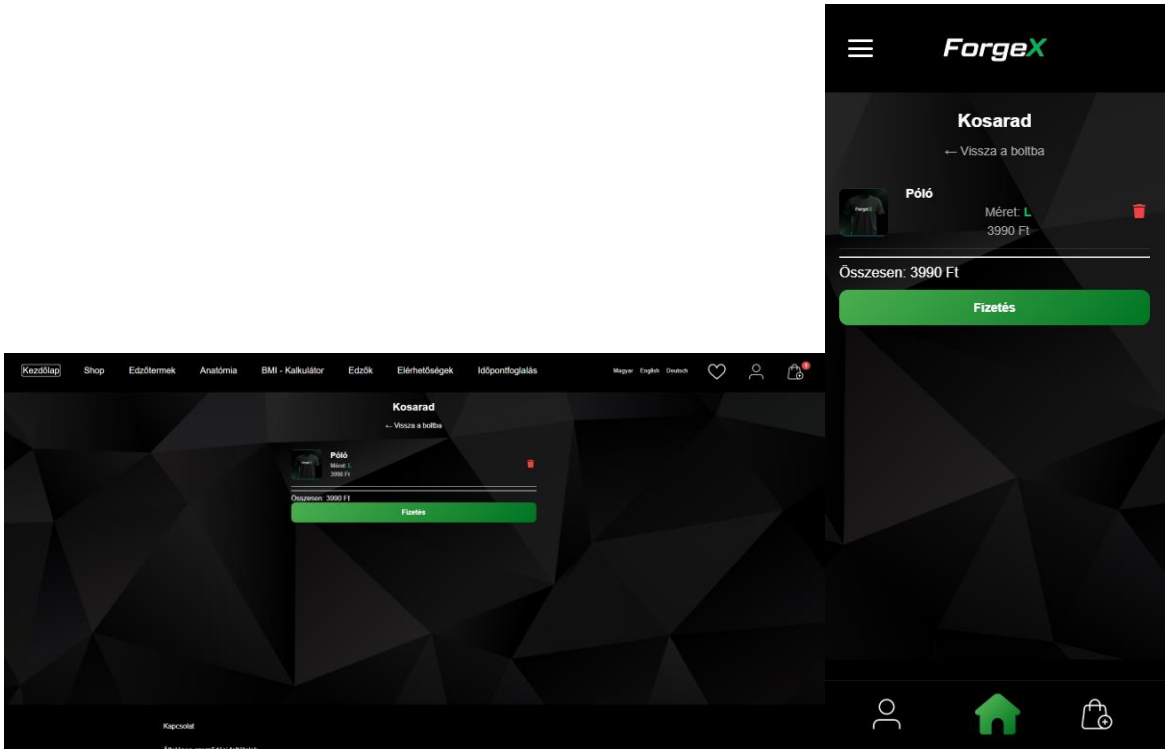
2.1. Felületi struktúra (cart.html)

A felület a letisztult **Checkout-orientált** designt követi:

- **Cart-Items-Container:** Egy dedikált #cart-items-container azonosítójú elem, amelybe a JavaScript dinamikusan injektálja a termékeket.
- **Summary Section:** A kosár alján elhelyezkedő összefoglaló sáv, amely a végösszeget (#total-price) és a továbblépési lehetőséget tartalmazza.

2.2. Design és Reszponzivitás (cart.css)

- **Mobile-First megközelítés:** Mobilon a termékinformációk sűrítve jelennek meg, a törlés gomb pedig könnyen elérhető méretű.
- **Interaktivitás:** A .delete-btn hover effektusa (piros kiemelés) egyértelmű jelzést ad a felhasználónak a művelet súlyáról.
- **Üveg-effektus (Glassmorphism):** A kártyák és inputok illeszkednek a ForgeX globális vizuális nyelvéhez (style.css változók használata).



A kosár oldal mobilon és asztali számítógépen

3. Programozási logika (cart.js)

3.1. Adat-szinkronizációs folyamat

A modul legfontosabb része a Firebase és a böngésző közötti kapcsolat kezelése:

1. **Auth Figyelő:** Az onAuthStateChanged belépéskor azonnal lekéri a Firestore-ból a carts gyűjtemény adott UID-jához tartozó dokumentumát, és frissíti a helyi tárolót.
2. **Törlés (removeFromCart):** Ez a funkció globálisan elérhető (window objektum), így a dinamikusan generált HTML-ből is hívható.

3.2. Dinamikus Renderelés (renderCart)

Ez a függvény felelős a UI konzisztenciájáért:

- **Kalkuláció:** Egy forEach ciklus segítségével összegzi a termékek árait (totalPrice).
- **Szelektív megjelenítés:** Ha a kosár üres, a "Pénztár" gombot elrejti (display: none), és információs szöveget jelenít meg.
- **Formázás:** A toLocaleString('hu-HU') metódussal biztosítja a magyar pénznem formátumot (szóköz ezres elválasztóként).

4. Tesztelési napló

ID	Tesztelt eset	Művelet	Elvárt eredmény	Állapot
TC-C01	Kosár listázás	Oldal megnyitása 2 termékkel a localStorage-ban.	Mindkét termék megjelenik helyes adatokkal és a helyes végösszeggel.	**Megfelelt**
TC-C02	Termék törlése	A kuka ikonra kattintás egy tételnél.	A tétel eltűnik, a végösszeg csökken, a Firestore-ban az items tömb frissül.	**Megfelelt**
TC-C04	Felhő szinkron	Bejelentkezés egy másik böngészőből.	A korábban (másik gépen) kosárba tett termékek megjelennek a Firestore-ból.	**Megfelelt**

Fizetés

1. Funkcionális leírás

A Fizetési modul célja a vásárlási folyamat lezárása. Itt történik a szállítási és számlázási adatok bekérése, a kosár tartalmának végső ellenőrzése, valamint a fizetési adatok validálása.

Főbb funkciók:

- **Rendelés összesítő:** A localStorage-ból kinyert tételek és a végösszeg dinamikus megjelenítése a fizetés előtt.
- **Komplex Validáció:** Szállítási (név, email, cím) és bankkártya adatok (16 számjegy, lejárat, CVV) formai ellenőrzése.
- **Maszkolt bevitel:** Automatikus formázás a kártyaadatoknál (pl. MM/YY perjel beszúrása).
- **Tranzakció utáni logika:** Sikeres fizetés esetén a kosár ürítése a felhőből (Firebase).
- **Vizuális visszajelzés:** Siker üzenet megjelenítése és a vásárlási űrlap eltüntetése a folyamat végén.

2. Frontend felépítés (HTML & CSS)

2.1. Felületi felosztás (pay.html)

A fizetési oldal három fő logikai szekcióra oszlik:

1. **Kosár tartalma:** Dinamikusan generált lista a rendelt tételekről.
2. **Szállítási adatok:** Felhasználói információk bekérése.
3. **Fizetési adatok:** Bankkártya mezők specifikus korlátozásokkal.

2.2. Design és UX (pay.css)

- **Glassmorphism:** A .payment-card egységes vizuális stílust képvisel a többi modullal.
- **Hibaállapotok:** Az .input-error osztály piros keretet ad a hibás mezőknek, az .error-message pedig szöveges segítséget nyújt.
- **Responsive Layout:** Mobilon a kétoszlopos űrlapmezők (pl. Város/Irányítószám) egymás alá rendeződnek a kényelmes gépelés érdekében.

3. Programozási logika (pay.js)

3.1. Validációs mechanizmus

A paymentForm.addEventListener('submit', ...) blokkban egy egyedi validációs motort implementáltam:

- **Reguláris kifejezések (Regex):** Az email formátumot, a 4 jegyű irányítószámot és a 16 jegyű kártyaszámot regex-szel ellenőrzi a rendszer.
- **Valós idejű szűrés:** A számjegyeket váró mezőknél (cardNumber, cvv, zip) a replace(/\D/g, '') parancs azonnal eltávolítja a betűket gépelés közben.

3.2. Kosár ürítése (clearCart)

A sikeres fizetés után kritikus fontosságú az adatok takarítása:

1. **Firebase:** A setDoc segítségével a carts gyűjteményben az adott felhasználóhoz egy üres tömböt mentünk le, így minden eszközén frissül az állapot.
2. **UI Frissítés:** A fejlécben lévő számláló (cart-count) nullázása.

4. Tesztelési napló

ID	Tesztelt eset	Művelet	Elvárt eredmény	Állapot
TC-P01	Üres mező validáció	Fizetés gomb megnyomása adatok nélkül.	Minden kötelező mező pirosra vált, hibaüzenetek megjelennek.	**Megfelelt**
TC-P02	Kártyaszám maszkolás	Betűk beírása a kártyaszám mezőbe.	A rendszer nem engedi a betűk megjelenését, csak számokat.	**Megfelelt**
TC-P04	Sikeres fizetés	Minden mező helyes kitöltése + Submit.	Az űrlap eltűnik, megjelenik a "Rendelését rögzítettük" doboz.	**Megfelelt**
TC-P05	Utólagos ellenőrzés	Fizetés után a Kosár oldal megnyitása.	A kosár üresnek látszik, a számláló 0-t mutat.	**Megfelelt**

Adatbázis

firebase.js – Infrastruktúra és Konfiguráció

Ez a fájl az alkalmazás központi kapcsolati pontja a Firebase felhőalapú szolgáltatásaival. Feladata az SDK inicializálása, a globális konfigurációs adatok tárolása, valamint a Firebase-szolgáltatások (Authentication és Firestore) elérhetővé tétele a többi modul számára. (Az inicializálás az a folyamat, amely során a projekt konfigurációs adatokkal létrehozzuk a Firebase-kapcsolatot, így a szolgáltatások (hitelesítés, adatbázis) elérhetővé és használhatóvá válnak az alkalmazás számára.)

A modul főbb feladatai:

1. **Firebase SDK inicializálása:** A fájl a Firebase **Modular SDK (v9+)** verzióját használja. Ez a megközelítés lehetővé teszi a *tree-shaking* alkalmazását, így az alkalmazás csak azokat a kódrészeket tölti be, amelyeket ténylegesen használunk, jelentősen csökkentve ezzel a weboldal betöltési idejét.

2. **Konfigurációs Objektum (firebaseConfig):** Ez az objektum tartalmazza a projekt egyedi azonosítóit (API Key, Project ID, stb.). Ezek biztosítják, hogy az alkalmazás a megfelelő Firebase-projektbe küldje az adatokat és onnan kérje le azokat.

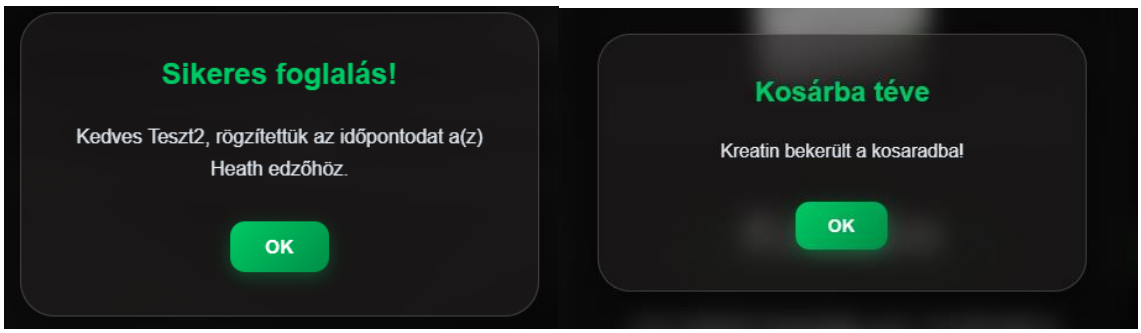
3. **Szolgáltatások központosítása és exportálása:** A getAuth(app) és getFirestore(app) inicializálása után a db és auth objektumokat exportáljuk. Ez biztosítja az **egységes forrást (Single Source of Truth)**: az összes többi modul (mint a cart.js vagy favourite.js) ugyanazt a példányt használja, így nem jönnek létre redundáns kapcsolatok a szerverrel.

Technikai részletek:

- **Függőségek:** A firebase-app.js, firebase-auth.js és firebase-firestore.js importálása biztosítja a szükséges funkciókat.
- **Eseménykezelés:** Mivel a fájl csak az inicializálást végzi, nincsenek benne DOM-manipulációs logikák, ami tisztán tartja a függőségi gráfot.

utils.js – Segédfüggvények és UI Komponensek

A utils.js egy központi tárolóhely az alkalmazásban gyakran használt segédfüggvények számára. Jelenleg egyetlen, de kulcsfontosságú elemet tartalmaz: a forgeXModal függvényt. Ez a modul felel azért, hogy egységes, a webshop arculatához illeszkedő vizuális visszajelzést (modális ablakot) nyújtson a felhasználónak, kiváltva a böngésző alapértelmezett, nem testreszabható alert() vagy confirm() dobozait.



Személyre szabott üzenetek

A modul főbb jellemzői:

1. **Dinamikus DOM-manipuláció:** A függvény nem a HTML-ben előre megírt elemeket használja, hanem a hívás pillanatában hozza létre őket a document.createElement segítségével. Ez a "tiszta" módszer: amíg nincs szükség rá, nem terheli a DOM-ot, majd a művelet végeztével el is távolítja az elemeket (overlayElement.remove()).
2. **Promise-alapú aszinkron működés:** A függvény egy Promise-t (ígéretet) ad vissza. Ez azért zseniális, mert lehetővé teszi, hogy a kódod "megvárja", amíg a felhasználó rákattint az OK vagy Mégse gombra.
    - *Példa:* const confirmed = await forgeXModal(...) Ez az async/await minta modern, olvasható és szinkron logikát tesz lehetővé egy alapvetően aszinkron felhasználói interakciónál.
3. **Testreszabhatóság:** A függvény paraméterei (title, message, isConfirm) lehetővé teszik, hogy ugyanazt a logikát használd egy egyszerű figyelmeztetésre (Alert) és egy döntést igénylő párbeszédpanelre (Confirm) is.

Kódmagyarázat:

- **HTML Injektálás:** A template literálok (`${...}`) használatával a dinamikus tartalom könnyen beilleszthető a modal dobozba.
- **Animáció kezelés:** A `setTimeout` használata biztosítja a CSS `active` osztály hozzáadását a létrehozás utáni pillanatban, így a böngésző tudja animálni a megjelenést (fade-in hatás).
- **Tisztítás (Cleanup):** A `resolve` meghívása után a `setTimeout` gondoskodik a modal eltávolításáról, ami memóriakezelési szempontból is példaértékű.

auth-header.js – Dinamikus Navigáció és Állapotkezelés

Ez a fájl egy ún. **Observer (megfigyelő) mintát** valósít meg, amely figyeli a felhasználói autentikációs állapotot, és ennek megfelelően módosítja a weboldal navigációs elemeit (pl. a fejléc vagy az alsó navigációs sáv linkjeit). Ez a fájl teszi lehetővé hogy a profil oldalt csak a bejelentkezett felhasználók láthassák és amíg nincs bejelentkezve addig az ember ikon a jobb felső sarokban a bejelentkezés oldalra viszi a felhasználót de amint megtörtént a bejelentkezés ugyanez a gomb már a profil oldalra irányít ahol látni lehet a felhasználó adatait valamint a foglalásokat a különféle órákra.

A modul főbb feladatai:

1. **Állapotfigyelés (onAuthStateChanged):** A fájl a Firebase onAuthStateChanged metódusát használja, amely egy "figyelő" (listener). Ez a függvény azonnal lefut, amint a felhasználó bejelentkezik vagy kijelentkezik. Így az alkalmazás képes valós időben reagálni a státuszváltásra anélkül, hogy az oldalnak újra be kellene töltődnie.

2. **DOM Szinkronizáció:** A querySelectorAll segítségével a kód megkeres minden olyan linket (signIn.html vagy #profile-link), amely a beléptetéssel kapcsolatos. Amint az autentikációs állapot megváltozik, a link.href tulajdonságát átírja:
    - **Bejelentkezett állapotban:** A hivatkozások a profil.html-re mutatnak.
    - **Kijelentkezett állapotban:** A hivatkozások a signIn.html-re mutatnak.

3. **Egységes élmény:** Mivel a kód egyszerre kezeli a fejlécet és az alsó navigációs menüt, a felhasználó bárhol is kattint az alkalmazásban, mindig a státuszának megfelelő oldalra jut.

Kódmagyarázat:

- **DOMContentLoaded:** A kód biztosra megy, és csak azután fut le, miután a HTML struktúra már betöltődött a böngészőbe. Ez megelőzi a "null pointer" hibákat (amikor a JavaScript megpróbál manipulálni egy elemet, ami még nem is létezik).
- **Rugalmasság:** A querySelectorAll használata különösen professzionális, mert így a kód akkor is működni fog, ha a jövőben újabb navigációs elemeket adunk az oldalhoz – a JS automatikusan megtalálja és felruházza őket a dinamikus linkelési logikával.

Tesztelési napló

Tesztelés tárgya	Elvárt eredmény	Tényleges eredmény	Státusz
firebase.js inicializálás	A Firebase app, auth és db objektumok sikeresen inicializálódnak, nincs konzolhiba.	A Firebase szolgáltatások elérhetőek a konzolban.	**Megfelelt**
utils.js Promise működés	Az OK/Mégse gombra kattintva a Promise resolve (lefut) és az ablak bezárul.resolve (lefut) és az ablak bezárul.	async/await hívások a kódban helyesen várnak az eredményre. helyesen várnak az eredményre.	**Megfelelt**

utils.js DOM tisztítás	Az ablak bezárása után a div eltűnik a DOM-ból.	Nincs visszamaradt elem a dokumentum végén.	**Megfelelt**
utils.js Modális megjelenés	A forgeXModal hívásakor a DOM-ba bekerül az overlay és a tartalom.	Overlay megjelenik az oldalon a megfelelő tartalommal.	**Megfelelt**
utils.js DOM tisztítás	Az ablak bezárása után a div eltűnik a DOM-ból.	Nincs visszamaradt elem a dokumentum végén.	**Megfelelt**
auth-header.js kijelentkezés	Kijelentkezett állapotban a linkek signIn.html-re mutatnak.	A linkek átirányítása kijelentkezett állapotban helyes.	**Megfelelt**
auth-header.js bejelentkezés	Bejelentkezett állapotban a linkek profil.html-re mutatnak.	A linkek átirányítása bejelentkezett állapotban helyes.	**Megfelelt**